CogniSight

What to do next on the Pi

A practical guide for Jahnvi and the contributor with the physical devices.

Start here	Laptop and Pi on the same Wi-Fi. SSH user: jahnvi. Hostname: cognisight.local. The IP and installed OS release still need checking.
Software prepared	Recognition/context, verified GPSD location/status, caregiver monitoring, module contracts, native diagnostics and software integration checks. Read docs/VALIDATION.md for exact results.
Camera first	Confirm the 22-pin Pi / 15-pin camera cable with power disconnected. Identify the real sensor, take a private test picture and check visible preview.
Audio later	The owned headset is AUX. Select a compatible USB sound adapter with microphone input when needed. Do not expect the Pi 5 to have an analogue headset jack.
GNSS stays off	The reported SIMA7670C/RS232 board is not proven to support GNSS. Identify its exact manual and electrical interface before connecting it. Do not wire RS232 to GPIO.
Phone last	Start one working app, then configure Caddy HTTPS and trust only its verified public root certificate. Never bypass a certificate warning.

How to use this guide

Every numbered step says where to type, what to expect and what to do if it fails. Stop at a failed checkpoint. Device instructions are reviewed procedures; they have not been executed on this Pi. Indoor navigation remains deferred.

The following pages reproduce the maintained SSH runbook. Links refer to the repository and official sources. Keep passwords, face samples, recordings, locations and private keys out of shared files.

Run CogniSight on your Pi using SSH

Software procedure ready; physical checks pending. Run these steps on the physical Pi and record what actually happens. Commands and repository paths were reviewed on the development computer; these steps have not been executed on your Pi.

Your Pi is already set up with Raspberry Pi OS 64-bit, user **jahnvi**, and hostname **cognisight**. Keep its existing OS and files. Keep your laptop and Pi on the same Wi-Fi. Commands below use a Mac/Linux terminal; Windows commands need to be adapted first.

You have an **AUX headset**; audio setup is for later. Pi 5 has no headset jack, so the current app needs a suitable **USB sound adapter**, including microphone input if your headset has a mic. Camera checks can proceed first; headset tests and app startup wait for valid audio settings.

SSH lets your laptop open a terminal on the Pi. **On your laptop** means a local terminal; **On your Pi** means the terminal opened through SSH. Keep passwords and private files on your devices, out of chat and Git. Replace any `REPLACE_...` text with your own value.

1. Connect to the Pi

On your laptop:

```
ssh jahnvi@cognisight.local
```

On the first connection, verify the displayed key fingerprint against the Pi's local screen or a trusted saved record before accepting it; see the appendix if needed.

Expected: You reach a terminal for `jahnvi` on the Pi. **If it fails:** Check power, Wi-Fi and SSH; if the name is not found, use the IP fallback in the appendix.

2. Check the Pi before installing anything

On your Pi:

```
cat /etc/os-release
uname -m
python3 --version
df -h .
```

The planned starting point is **Bookworm, Python 3.11, aarch64**; it has not yet been tested on your Pi. The installed camera libraries and Python packages must match.

Expected: You can identify the OS, Python version and free space. **If it differs or is unclear:** Stop before installing anything; review a matching setup for that OS, without erasing the card or downgrading Python.

3. Fit the camera with power off

On your Pi: Stop CogniSight if it is running, then shut down.

```
sudo shutdown -h now
```

Wait for shutdown and unplug power. Pi 5 uses a **22-pin camera connector**; the intended Camera Module 2 uses a **15-pin connector**. Your photos show **Camera V2.0**, cable **HBV-Raspberry15DFPC** and 22 visible contacts at the free end; the camera end is hidden, so confirm the cable's fit before connecting it. The labels do not prove an official camera or its sensor. Follow the official cable instructions. Check the power supply and cooling, reconnect power, then repeat Step 1. Do not wire the SIM A7670C/RS232 board yet.

Expected: The camera is connected and the Pi boots again. **If it fails:** Check the cable and power with the Pi unplugged; never move the camera ribbon while powered.

4. Copy the approved local code

On your laptop, in another terminal: Use your actual checkout path; the path below is the known workspace example.

```
cd /Users/acctest/cognisight/CogniSight
git status --short
git rev-parse HEAD > /tmp/cognisight-commit.txt
git archive --format=tar.gz --output=/tmp/cognisight-code.tar.gz HEAD
scp /tmp/cognisight-code.tar.gz /tmp/cognisight-commit.txt jahnvi@cognisight.local:/tmp/
```

Run this only after the approved changes are committed locally and committed files have been checked for private data. This transfers the exact committed version you reviewed, regardless of whether GitHub main has moved. This archive copies committed files without old Git history, ignored models or private state; uncommitted edits are excluded.

Expected: Both files reach the Pi; the text file records which commit was copied. **If it fails:** Check the checkout path and SSH connection; finish intended local changes before transfer.

5. Unpack into a new folder

On your Pi:

```
COGNI_REV=$(cat /tmp/cognisight-commit.txt)
COGNI_BUILD="$HOME/cognisight-builds/$COGNI_REV"
mkdir -p "$HOME/cognisight-builds"
mkdir "$COGNI_BUILD" &&
tar -xzf /tmp/cognisight-code.tar.gz -C "$COGNI_BUILD" &&
cp /tmp/cognisight-commit.txt "$COGNI_BUILD/.source-commit" &&
cd "$COGNI_BUILD" &&
cat .source-commit
```

Expected: A new folder contains the copied code and its commit number, leaving existing installs and state untouched. **If it says the folder exists or extraction fails:** Stop and inspect it; do not unpack over an existing install.

6. Prepare Python

On your Pi: Continue only after Step 2's OS check confirms the candidate baseline and the matching package setup is reviewed; internet is needed for installation and model downloads.

```
sudo apt update
apt-cache policy python3-picamera2 python3-venv libportaudio2
sudo apt install python3-venv python3-picamera2 libportaudio2
python3 -m venv --system-site-packages .venv
. .venv/bin/activate
python -m pip install --dry-run -e '.[vision,audio,dev]'
```

Expected: OS-matched camera packages install, and pip shows a package list without conflicts for review before the real install. **If it fails or proposes building camera/numerical libraries from source:** Stop and review the package versions; do not use sudo pip or install libcamera through pip.

7. Install and check the Python packages

On your Pi: After reviewing Step 6's package list:

```
python -m pip install -e '.[vision,audio,dev]'
python -m pip check
cognisight doctor
cognisight doctor --native
python - <<'CHECK'
import numpy, cv2, libcamera
from picamera2 import Picamera2
print("NumPy", numpy.__version__)
print("OpenCV", cv2.__version__)
CHECK
cognisight devices
```

Expected: The Python checks finish without errors; audio remains pending until your USB sound adapter is connected and listed. **If it fails:** Check matching camera/Python packages, and leave missing audio pending without switching to a default speaker.

8. Take a camera test picture

On your Pi: Stop CogniSight and other camera programs first; point at a scene without people.

```
rpicam-hello --list-cameras
umask 077
COGNI_CAMERA_DIR=$(mktemp -d /tmp/cognisight-camera.XXXXXX)
rpicam-still --nopreview --timeout 2000 -o "$COGNI_CAMERA_DIR/check.jpg"
printf '%s\n' "$COGNI_CAMERA_DIR"
```

Expected: The actual camera sensor is listed, a picture is saved, and its folder path is printed. **If it fails:** Check whether another program owns the camera; if no camera is found, shut down and unplug power before checking the cable.

9. Look at the picture

On your laptop: Replace `REPLACE_CAMERA_DIR` with the full folder path printed in Step 8, then open the copied picture in your usual image viewer.

```
scp jahnvi@cognisight.local:REPLACE_CAMERA_DIR/check.jpg /tmp/cognisight-check.jpg
```

Expected: You see a clear picture of the test scene. **If it is missing or unusable:** Fix capture, lighting or focus before moving on.

10. Delete the test picture

On your Pi, in the same SSH terminal as Step 8: Also delete `/tmp/cognisight-check.jpg` on your laptop using its file manager.

```
rm "$COGNI_CAMERA_DIR/check.jpg"
rmdir "$COGNI_CAMERA_DIR"
```

Expected: The private test picture is removed from both devices. **If it fails:** Check the exact path and remove only the test file; do not use broad delete commands.

11. Check visible camera preview

On your Pi, in its own desktop terminal with a monitor attached:

```
rpicam-hello --timeout 5000
```

Expected: You see a live preview for five seconds. **If no monitor is available:** Record preview as pending; an SSH capture with no preview does not prove this step passed.

12. Download models; set audio later

On your Pi, in the copied build folder with `.venv` active:

```
python scripts/fetch_models.py --destination models
cp -n config.example.json config.json
chmod 600 config.json
cognisight devices
sha256sum models/face_recognition_sface_2021dec.onnx
nano config.json
```

Set `model_version` to `sface:` followed by the printed hash, matching `models/manifest.json`. Keep `calibrated: false`, navigation disabled and caregiver access disabled for now. `cp -n` preserves an existing config. When your sound adapter is ready, enter its exact USB input/output names; until then, leave audio unconfigured and skip Steps 13-15.

Expected: Models download successfully and the real hash is recorded, with audio settings completed only after the adapter is ready. **If downloads, hashes or later audio checks fail:** Stop and fix them; do not turn off model checks or use speaker fallback.

13. Run a short camera and headset check

On your Pi, after the USB sound adapter and microphone are configured: Keep CogniSight stopped; wear the headset, answer the playback prompt and speak "Computer, who is this?" when asked.

```
mkdir -p qualification-private
chmod 700 qualification-private
python scripts/qualify.py --output qualification-private/inventory.json
python scripts/qualify.py --config config.json --seconds 10 \
  --interactive --output qualification-private/smoke.json
```

Expected: A private report records camera frames, measured processing time and your actual headset check. `recognizer_features: pending` means no accepted face feature was measured (for example, an empty scene). `smoke_passed` on that field proves feature execution only, never correct identity recognition. **If it fails:** Read the report's failed step and fix that device/model problem; ten seconds passing is not proof of recognition accuracy.

The first command saves inventory separately. The ten seconds is a sample period, not protection from a stuck camera library: if native code hangs or crashes, the smoke report may not be written, while the earlier inventory remains.

14. Start CogniSight

On your Pi: Set the caregiver password privately at the prompts, then start one app.

```
cognisight password
cognisight run --config config.json
```

Expected: One app runs; Ctrl-C stops it and releases the devices. **If it fails:** Check config, models and headset; do not start another copy or run the app as root.

15. Open local enrollment from your laptop

On your laptop, in another terminal: Keep this connection open, then visit `http://127.0.0.1:8875` in your laptop browser.

```
ssh -N -L 127.0.0.1:8875:127.0.0.1:8765 jahnvi@cognisight.local
```

Sign in using the password from Step 14. This private connection keeps enrollment off the Wi-Fi network. Close it with Ctrl-C when finished. Follow the recognition guide before enrolling consenting people or enabling named announcements; do not change `calibrated` just to force a demo.

Expected: The local enrollment page opens and accepts your login. **If local port 8875 is busy:** Choose another free local port in both the command and browser URL.

16. Run the software checks on this Pi

On your Pi, in the copied build folder with `.venv` active:

```
PYTHONPATH=src python -m unittest discover -s tests -v
python -m pip check
```

If Node is installed, also run `node --test tests/dashboard.test.cjs`. These are software checks; they do not replace the real camera, headset or phone steps. See validation for the development results to compare with this run.

Expected: No failed checks. Any skip is listed with its actual reason. **If it fails:** Save the test name and short error class privately. Fix that problem before enabling automatic startup; do not hide skips or change tests to obtain a pass.

17. Make the tested app a user service

Do this only after foreground startup and Ctrl-C both work. Stop the foreground app first. Keep the same build folder and private state; do not run two copies.

On your Pi, in the copied build folder: This prepares a unit with the real build path instead of the template's example path.

```
mkdir -p "$HOME/.config/systemd/user"
python - <<'UNIT'
from pathlib import Path
build = Path.cwd().resolve()
unit = build / 'deploy/cognisight.service'
target = Path.home() / '.config/systemd/user/cognisight.service'
if target.exists():
    raise SystemExit('A unit already exists. Back it up and review it before replacement.')
target.write_text(unit.read_text().replace('%h/CogniSight', str(build)))
UNIT
systemctl --user daemon-reload
systemctl --user start cognisight.service
systemctl --user status cognisight.service --no-pager
```

Expected: The status says `active (running)` and the local enrollment page works. This means the process runs, not that every module is qualified. The unit waits up to 90 seconds for shutdown; stuck native code can still need supervisor termination. **If it fails:** Run `journalctl --user -u cognisight.service -n 40 --no-pager`. Inspect it privately; share only non-identifying status/error classes. Check the unit's real paths and ordinary-user camera/audio permissions. Do not switch the runtime to root or make device files world-writable.

After the service passes the headset and camera checks, **on your Pi:**

```
systemctl --user enable cognisight.service
systemctl --user stop cognisight.service
systemctl --user start cognisight.service
systemctl --user restart cognisight.service
```

The four commands enable startup for user sessions, then check stop/start/restart. Confirm each operation before issuing the next. For startup after boot without logging in, first verify that headless audio works for `jahnvi`. Then choose to keep that user's services running after logout with `sudo loginctl enable-linger jahnvi`. This is an operating-system setting, not a substitute for device permissions. Reboot and repeat SSH, status, camera and audio checks. If headless audio fails, leave automatic startup disabled with `systemctl --user disable --now cognisight.service` until the route works.

18. Put the caregiver dashboard on HTTPS

This step needs the actual Pi IP and the caregiver's phone. Keep navigation disabled while the SIM A7670C board remains unverified. The dashboard can honestly show outdoor location as unavailable.

On your Pi: Find the LAN address with `hostname -I`. Pick one stable private address for this Wi-Fi; a router reservation helps. Choose a hostname the phone can actually resolve, or use that reserved IP. In `config.json`, set `caregiver.enabled` to `true`, set `allowed_hosts` to that exact hostname/IP, and keep backend port `8766`. Restart the app after saving. Do not put a scheme, URL path or port into `allowed_hosts`.

Install Caddy for the actual OS using its official installation instructions. Then inspect the installed service before copying configuration:

```
caddy version
systemctl show caddy -p User -p ExecStart -p FragmentPath
cp -n deploy/Caddyfile.example deploy/Caddyfile
chmod 600 deploy/Caddyfile
nano deploy/Caddyfile
```

In the copied Caddyfile, replace `{$COGNISIGHT_HOST}` with the chosen hostname/IP and `{$COGNISIGHT_LAN_IP}` with the Pi's actual private LAN IP. These are configuration placeholders, not passwords. Keep the upstream `127.0.0.1:8766`, the LAN-only matcher, internal TLS and disabled HTTP redirects. The file is ignored by Git.

If the installed Caddy unit uses `/etc/caddy/Caddyfile` and has no other site configuration to preserve, first back up that file privately. Then validate/install the reviewed file:

```
sudo caddy validate --config "$PWD/deploy/Caddyfile" --adapter caddyfile
sudo cp -n /etc/caddy/Caddyfile /etc/caddy/Caddyfile.cognisight-before
sudo install -o root -g root -m 644 deploy/Caddyfile /etc/caddy/Caddyfile
sudo systemctl reload caddy
systemctl status caddy --no-pager
```

Run each command only after the previous one succeeds. If the service is not running, use `sudo systemctl start caddy` after validation instead of reload. If its config path/account differs, use the actual installed service configuration; do not overwrite unrelated sites. Caddy's service guide explains those paths and commands.

Expected: App backend stays on loopback; Caddy listens for HTTPS on the chosen Pi LAN IP. Phone login is not ready until certificate trust is complete. **If it fails:** Check matching hostname/IP, Caddy validation, service status and app status. Never proxy enrollment `8765`, expose GPSD, add router port forwarding or disable certificate checks.

For the official `caddy` service account with default storage, the public CA certificate is normally `/var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt`. Verify the account/storage first; custom installations differ. **On your Pi:**

```
COGNI_CA=/var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt
sudo openssl x509 -in "$COGNI_CA" -noout -subject -fingerprint -sha256
```

For that verified default storage path, stage only the public certificate. **On your Pi:**

```
sudo install -o jahnvi -g "$(id -gn jahnvi)" -m 644 \
  "$COGNI_CA" /tmp/cognisight-root.crt
```

On your laptop:

```
scp jahnvi@cognisight.local:/tmp/cognisight-root.crt /tmp/cognisight-root.crt
openssl x509 -in /tmp/cognisight-root.crt -noout -fingerprint -sha256
```

Compare that fingerprint with the trusted Pi output, then transfer the public certificate to the phone through a trusted local method such as USB or AirDrop. Delete the temporary public-certificate copies after installation. Never change the CA directory's permissions or transfer `root.key`, private keys or the entire Caddy data directory.

On the phone: Install the public root using the phone's supported certificate process. For iPhone, a manually installed profile also needs explicit trust enabled in certificate trust settings; use Apple's instructions. On Android, use the manufacturer's certificate/CA installation settings; menus differ by version, and a Wi-Fi-only certificate entry may not establish browser CA trust. Use the actual phone's official instructions and verify in its browser. Do not choose a setting that skips validation.

Open `https://YOUR_CHOSEN_HOST/dashboard`, replacing `YOUR_CHOSEN_HOST` with the exact chosen name/IP. There must be no certificate warning. Sign in, check statuses, log out and confirm private values disappear. Confirm the fifteen-minute session expires, source values expire after their source stops, and reconnection requires new evidence. Indoor must remain not integrated. Keep the phone on the same private Wi-Fi; guest client isolation may block access.

19. Record physical acceptance and recovery

On the Pi and phone: Work through recognition qualification, GNSS checks once the board is identified, and caregiver acceptance. Record real OS/package/model versions, camera sensor, sound adapter/rates, power/cooling, actual test conditions and failures. Keep biometric samples, voices, real profiles/locations and private reports out of Git.

Check one combined run, internet-disconnected operation while local Wi-Fi remains available, source loss/recovery, queued-speech expiry, stop while speaking, logout/reboot and the two-hour thermal/memory/latency gates. A healthy process and a short smoke test do not prove these passed.

For rollback, **on your Pi:** stop the app service, confirm the process/device owners exited, and point the user unit back to a previously verified build/environment. Keep the private state and its reviewed backup; do not restore an unreviewed database over current profiles or install native packages under a running process. Disable the optional dashboard/Caddy service if only phone setup is failing. Use `systemctl --user status cognisight.service --no-pager` after restarting the selected known-good build. Do not delete locks blindly or run a second camera process to bypass a failed stop.

Expected: Every acceptance row has a real result or an explicit pending reason, and recovery starts one known version without losing private data. **If any result is unclear:** Leave that row pending. Software can be complete while device qualification remains incomplete.

Reference notes

These notes support the numbered procedure. Anything untested stays **pending**.

Connection help

On your laptop: If `cognisight.local` is not found, get the Pi's IP from your router's device list, or run `hostname -I` on the Pi's local screen; then connect with `ssh jahnvi@REPLACE_PI_IP`. Replace `cognisight.local` with that verified IP in later ssh/scp commands too.

For a first-connection fingerprint, **on the Pi's local screen** run `ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub` and compare it with SSH's prompt. If a previously saved key changes, stop and explain the change before updating it. See official remote-access help.

Package checks and software tests

The project candidate baseline is Bookworm/Python 3.11; latest Raspberry Pi OS downloads use Trixie, which needs its own check. The project currently uses NumPy below 2 and OpenCV 4.10 or newer/below 5. Camera libraries installed by apt and Python packages must work together. Use one intended OpenCV pip package; do not replace OS camera libraries or bypass OS Python protections. Plain doctor finds packages; it does not prove they load or the devices work. `cognisight doctor --native` actually imports required packages in separate child processes, with a ten-second timeout per dependency group. It opens no devices. A failed or timed-out probe means imports are not ready; a passed probe still does not prove hardware works. Its exit code is 0 for all imports passed, 1 for a failed/missing/timed-out import, and 2 for a command/reporting error. Its sanitized results omit native stderr and local paths.

On your Pi: Run `PYTHONPATH=src python -m unittest discover -s tests -v`; if Node is already installed, also run `node --test tests/dashboard.test.cjs`. Keep every skipped or failing test visible. `scripts/qualify.py` exit 1 means a requested check failed; exit 2 means command/output trouble. An inventory-only report or empty camera scene does not prove face recognition ran successfully.

Save actual OS/image, Python, apt/pip versions, camera sensor, headset/rates, power/cooling and model hashes. Keep package lists private and review them before sharing: they may contain local paths or private download addresses. The model downloader records hashes; Vosk's first download still relies on trusting its official HTTPS source.

GNSS and caregiver phone

Your board is described as **SIM A7670C MODULE WITH RS232**. Its exact board manual, GNSS support and connection levels are still unverified; an RS232 connection must not be wired directly to Pi GPIO. The 19:54:15 photo shows Imager settings, not the board. Follow the GNSS guide only after identifying the hardware, without guessed pins or modem commands. GPSD alone owns the receiver and stays local to the Pi. No fix means unavailable, not invented coordinates; there are no route directions.

Follow the caregiver HTTPS guide for Caddy and the actual phone OS. Use the same exact hostname/IP in app settings, Caddy and the phone URL. The phone must trust Caddy's **public root certificate**, with its fingerprint verified through a trusted channel; never copy the CA private key or bypass a browser warning. Only dashboard backend `127.0.0.1:8766` is proxied. Enrollment `127.0.0.1:8765` stays local. Keep both devices on the same Wi-Fi; do not expose ports to the internet.

Start after reboot

Adapt `deploy/cognisight.service` to this build's real folder; its example `%h/CogniSight` path differs. Use an ordinary user and only the device permissions needed. Headless audio and user-service startup need testing after logout/reboot; follow the service guidance before enabling it. Stop the foreground copy first, and check exactly one app starts. Do not grant root or world-writable device access to fix failures.

App state lives in `~/.local/share/cognisight`, outside the build folder. Preserve it during updates, along with your private config/models. After checking the transfer, delete the temporary archive and commit-number file from `/tmp` on each device.

Measure before declaring the system ready

Follow the existing test requirements for held-out recognition accuracy, unknown faces, response/stop timing, real voice commands, gallery size, two-hour use, memory, offline use and device recovery. A short camera test proves none of these by itself. Keep local Wi-Fi for SSH/phone when checking operation without internet. Never unplug the camera ribbon while powered. Record failed tests, limits and missing measurements honestly.